

Python Programming Homework 2

Full name: Jorge Alfaro

FAU ID: Z23697022

Instructor: Dr. Ionut Cardei

Draft status

This report contains the solutions for Problems 1 through 5 and the Problem 3 extra-credit tests. Required screenshots and VSCode coding-agent evidence are still pending.

Before submission, insert authentic screenshots in the marked locations and regenerate the PDF from the final Word document.

Contents

Problem 1 Python file parsing

Problem 2 Comprehensions

Problem 3 Social network and extra credit

Problem 4 IMDB rankings

Problem 5 Weather station analyzer and unit tests

Implementation notes

The programs use Python 3.10 or newer and only standard-library modules, plus the supplied `testif` module. Problem 4 uses a `p4` filename because its `p3` filename in the handout would overwrite Problem 3.

Problem 5 parses the example date format with `datetime` and retains the original date strings. Statistics output uses the columns `station`, `min`, `max`, `mean`, with stations in lexicographic order and one decimal place.

Problem 1 Python file parsing

p1_Alfaro_Jorge.py

```
"""Problem 1: number source lines and extract Python functions."""

import ast
import io
from pathlib import Path
import tokenize

def line_number(input_file: str, output_file: str) -> None:
    """Copy a text file, prefixing every line with its 1-based number."""
    try:
        source, target = Path(input_file), Path(output_file)
        if source.resolve() == target.resolve() or (
            target.exists() and source.samefile(target)):
            raise ValueError("Input and output must be different files.")
        with source.open(encoding="utf-8") as reader:
            lines = reader.readlines()
        with target.open("w", encoding="utf-8") as writer:
            for number, line in enumerate(lines, 1):
                writer.write(f"{number}: {line}")
    except (OSError, UnicodeError, ValueError) as error:
        print(f"Could not number the file: {error}")
        raise

def parse_functions(filename: str) -> tuple[tuple[int, str, str, str], ...]:
    """Return functions sorted by name as (line, name, arguments, code).

    Include nested functions and methods. Keep docstrings and '#' inside
    strings; remove comment tokens and empty lines from the returned code.
    """
    try:
        with tokenize.open(filename) as reader:
            source = reader.read()
        functions = []
        for node in ast.walk(ast.parse(source, filename)):
            if not isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
                continue
            code = ast.get_source_segment(source, node)
            lines = code.splitlines(keepends=True)
            tokens = list(tokenize.generate_tokens(io.StringIO(code).readline))
            depth, start, end = 0, None, None
            offsets = [0]
            for line in lines:
                offsets.append(offsets[-1] + len(line))
            for token in tokens:
                if token.type == tokenize.COMMENT:
                    row, column = token.start
                    lines[row - 1] = lines[row - 1][:column] + "\n"
                if token.type == tokenize.OP and end is None:
                    if token.string == "(":
                        if start is None:
                            start = offsets[token.end[0] - 1] + token.end[1]
                            depth += 1
                        elif token.string == ")" and start is not None:
                            depth -= 1
                            if depth == 0:
                                end = offsets[token.end[0] - 1] + token.end[1]
                    else:
                        continue
            functions.append((start, lines, end, code))
    except (OSError, UnicodeError, ValueError) as error:
        print(f"Could not parse the file: {error}")
        raise
```

Problem 1 Python file parsing continued

p1_Alfaro_Jorge.py

```
        end = offsets[token.start[0] - 1] + token.start[1]
        arguments = code[start:end]
        cleaned = "\n".join(line.rstrip() for line in lines if line.strip())
        functions.append((node.lineno, node.name, arguments, cleaned + "\n"))
    return tuple(sorted(functions, key=lambda item: item[1]))
except (OSError, UnicodeError, SyntaxError, tokenize.TokenError) as error:
    print(f"Could not parse Python functions: {error}")
    raise

def main() -> None:
    """Demonstrate both functions using this source file."""
    print("Student: Jorge Alfaro | FAU ID: Z23697022 | Problem 1")
    source = Path(__file__).resolve()
    destination = str(source) + ".txt"
    line_number(str(source), destination)
    print(f"Numbered source saved to {destination}")
    print(parse_functions(str(source)))

if __name__ == "__main__":
    main()
```

Required screenshots

[Insert authentic program-output screenshots showing parts a and b. Your name or FAU ID must be visible when captured and must not be added to the images.]

Problem 2 Comprehensions

p2_Alfaro_Jorge.py

```
"""Problem 2: list and dictionary comprehensions."""

def main() -> None:
    """Display the answers for parts a through f."""
    print("Student: Jorge Alfaro | FAU ID: Z23697022 | Problem 2")

    # a. Include every ordered tuple of four distinct integers.
    equal_squares = [
        (a, b, c, d)
        for a in range(1, 11) for b in range(1, 11)
        for c in range(1, 11) for d in range(1, 11)
        if len({a, b, c, d}) == 4 and a * a + b * b == c * c + d * d
    ]
    print("a:", equal_squares)

    # b. Lowercase words whose original length is less than five.
    words = ['One', 'SEVEN', 'three', 'two', 'Ten']
    short_words = [(word.lower(), len(word)) for word in words if len(word) < 5]
    print("b:", short_words)

    # c. Abbreviate the middle name in each three-part name.
    names = ['Christopher Ashton Kutcher', 'Elizabeth Stamatina Fey']
    abbreviated = [
        f"{first} {middle[0]}. {last}"
        for name in names for first, middle, last in [name.split()]
    ]
    print("c:", abbreviated)

    # d. Compare sorted lowercase characters to identify anagrams.
    lst1 = ["Spam", "Trams", "Elbows", "Tops", "Astral"]
    lst2 = ["Bowels", "Sample", "Altars", "Stop", "Course", "Smart"]
    anagrams = [(w1, w2) for w1 in lst1 for w2 in lst2
                 if sorted(w1.lower()) == sorted(w2.lower())]
    print("d:", anagrams)

    # e. Map each distinct string to its length.
    s = ['one', 'two', 'three']
    lengths = {word: len(word) for word in s}
    print("e:", lengths)

    # f. Retain vowel positions and their original character case.
    text = "Hello world"
    vowels = {i: c for i, c in enumerate(text) if c.lower() in "aeiou"}
    print("f:", vowels)

if __name__ == "__main__":
    main()
```

Required screenshots

[Insert authentic program-output screenshots showing parts a through f. Your name or FAU ID must be visible when captured and must not be added to the images.]

Problem 3 Social network

p3_Alfaro_Jorge.py

```
"""Problem 3: a social network with CSV persistence and extra-credit tests."""

import csv
from pathlib import Path
from tempfile import TemporaryDirectory

from testif import testif

Network = dict[str, tuple[str, list[str]]]

def add_user(sn: Network, username: str, fullname: str) -> bool:
    """Add a user with no friends; return False for an existing username."""
    try:
        if username in sn:
            return False
        sn[username] = (fullname, [])
        return True
    except (TypeError, ValueError) as error:
        print(f"Could not add the user: {error}")
        raise

def add_friend(sn: Network, user1: str, user2: str) -> bool:
    """Add a mutual link; reject missing users, self-links and existing links."""
    try:
        if user1 not in sn or user2 not in sn or user1 == user2:
            return False
        if user2 in sn[user1][1] and user1 in sn[user2][1]:
            return False
        if user2 not in sn[user1][1]:
            sn[user1][1].append(user2)
        if user1 not in sn[user2][1]:
            sn[user2][1].append(user1)
        return True
    except (TypeError, KeyError, IndexError, AttributeError) as error:
        print(f"Could not add the friendship: {error}")
        raise

def get_friends(sn: Network, user1: str, distance: int) -> list[str]:
    """Return unique friends within distance links in breadth-first order."""
    try:
        if not isinstance(distance, int) or isinstance(distance, bool) or distance < 1:
            raise ValueError("Distance must be a positive integer.")
        if user1 not in sn:
            return []
        seen, frontier, result = {user1}, [user1], []
        for _ in range(distance):
            following = []
            for user in frontier:
                for friend in sn[user][1]:
                    if friend not in seen:
                        seen.add(friend)
                        following.append(friend)
                        result.append(friend)
            frontier = following
```

Problem 3 Social network continued

p3_Alfaro_Jorge.py

```
        if not frontier:
            break
        return result
    except (TypeError, ValueError, KeyError, IndexError) as error:
        print(f"Could not find friends: {error}")
        raise

def save_network(filename: str, sn: Network) -> None:
    """Save headerless CSV rows: username, full name, then each friend."""
    try:
        with open(filename, "w", newline="", encoding="utf-8") as file:
            writer = csv.writer(file)
            for username, (fullname, friends) in sn.items():
                writer.writerow([username, fullname, *friends])
    except (OSError, UnicodeError, csv.Error, TypeError, ValueError) as error:
        print(f"Could not save the network: {error}")
        raise

def load_network(filename: str) -> Network:
    """Read a network saved by save_network, rejecting malformed rows."""
    try:
        network = {}
        with open(filename, newline="", encoding="utf-8") as file:
            for row in csv.reader(file):
                if len(row) < 2 or row[0] in network:
                    raise ValueError("Missing fields or duplicate username.")
                network[row[0]] = (row[1], row[2:])
        return network
    except (OSError, UnicodeError, csv.Error, ValueError) as error:
        print(f"Could not load the network: {error}")
        raise

def test() -> None:
    """Test parts a-e with the supplied testif function (extra credit)."""
    try:
        sn = {}
        results = []
        for user, name in [("alice", "Alice Smith"), ("maria", "Maria Cortez"),
                           ("joe", "Joseph Adams"), ("eve", "Evelyn Cooper"),
                           ("david", "David Benson")]:
            results.append(testif(add_user(sn, user, name), f"add {user}"))
            results.append(testif(not add_user(sn, "alice", "Other"), "duplicate user"))
        for left, right in [("alice", "maria"), ("maria", "joe"),
                            ("joe", "eve"), ("maria", "david")]:
            results.append(testif(add_friend(sn, left, right), f"link {left}-{right}"))
            results.append(testif(not add_friend(sn, "alice", "missing"), "missing user"))
            results.append(testif(not add_friend(sn, "alice", "alice"), "self-link"))
            results.append(testif(not add_friend(sn, "alice", "maria"), "duplicate link"))
            results.append(testif(get_friends(sn, "alice", 1) == ["maria"], "distance 1"))
            results.append(testif(get_friends(sn, "alice", 2) == ["maria", "joe", "david"],
                                   "distance 2"))
            results.append(testif(get_friends(sn, "alice", 20) ==
                                   ["maria", "joe", "david", "eve"], "cycles"))
            results.append(testif(get_friends(sn, "unknown", 2) == [], "unknown user"))
```

Problem 3 Social network continued

p3_Alfaro_Jorge.py

```
add_user(sn, "solo", 'Solo, "Student"')
results.append(testif(get_friends(sn, "solo", 1) == [], "isolated user"))
with TemporaryDirectory() as folder:
    filename = str(Path(folder) / "network.csv")
    save_network(filename, sn)
    results.append(testif(Path(filename).is_file(), "save network"))
    results.append(testif(load_network(filename) == sn, "CSV round trip"))
if not all(results):
    raise AssertionError("A social network test failed.")
except (OSError, ValueError, TypeError, AssertionError) as error:
    print(f"Social network tests could not complete: {error}")
    raise

def main() -> None:
    """Run the tests for all social network operations."""
    try:
        print("Student: Jorge Alfaro | FAU ID: Z23697022 | Problem 3")
        test()
    except (OSError, ValueError, TypeError, AssertionError) as error:
        print(f"Problem 3 could not finish: {error}")
        raise

if __name__ == "__main__":
    main()
```

Required screenshots

[Insert authentic program-output screenshots showing parts a through g. Your name or FAU ID must be visible when captured and must not be added to the images.]

Problem 4 IMDB rankings

p4_Alfaro_Jorge.py

```
"""Problem 4: rank director-actor collaborations and actor box office."""

import csv
from pathlib import Path

DATA_DIRECTORY = Path(__file__).resolve().parent


def read_movies(filename: str, value_column: str) -> dict[tuple[str, str], float]:
    """Map (title, year) to a numeric value from a CSV with a header."""
    with open(filename, newline="", encoding="utf-8") as file:
        return {(row["Title"], row["Year"]): float(row[value_column])
                for row in csv.DictReader(file)}


def read_casts(filename: str) -> dict[tuple[str, str], tuple[str, list[str]]]:
    """Map (title, year) to (director, actors) from the headerless cast CSV."""
    with open(filename, newline="", encoding="utf-8") as file:
        return {(row[0], row[1]): (row[2], row[3:]) for row in csv.reader(file)}


def display_ranking(totals: dict, limit: int | None = None) -> None:
    """Print descending totals; None displays all and zero displays none."""
    if limit is not None and limit < 0:
        raise ValueError("The display limit cannot be negative.")
    ranking = sorted(totals.items(), key=lambda item: (-item[1], item[0]))
    for rank, (key, total) in enumerate(ranking[:limit], 1):
        if isinstance(key, tuple):
            print(f"rank:2}. {(key[0], key[1], total)}")
        else:
            print(f"rank:2}. {key}: ${total:,.0f}")


def display_top_collaborations(limit: int | None = None) -> None:
    """Rank director-actor pairs by number of shared top-rated movies."""
    rated = read_movies(str(DATA_DIRECTORY / "imdb-top-rated.csv"), "IMDb Rating")
    casts = read_casts(str(DATA_DIRECTORY / "imdb-top-casts.csv"))
    totals = {}
    for movie in rated:
        director, actors = casts[movie]
        for actor in set(actors):
            pair = (director, actor)
            totals[pair] = totals.get(pair, 0) + 1
    display_ranking(totals, limit)


def display_top_actors(limit: int | None = None) -> None:
    """Rank actors by the sum of their movies' top-grossing box office."""
    grossing = read_movies(str(DATA_DIRECTORY / "imdb-top-grossing.csv"),
                           "USA Box Office")
    casts = read_casts(str(DATA_DIRECTORY / "imdb-top-casts.csv"))
    totals = {}
    for movie, amount in grossing.items():
        for actor in set(casts[movie][1]):
            totals[actor] = totals.get(actor, 0) + int(amount)
    display_ranking(totals, limit)
```

Problem 4 IMDB rankings continued

p4_Alfaro_Jorge.py

```
def main() -> None:
    """Demonstrate both rankings, each limited to ten entries."""
    print("Student: Jorge Alfaro | FAU ID: Z23697022 | Problem 4")
    print("Top director-actor collaborations")
    display_top_collaborations(10)
    print("\nTop actors by USA box office")
    display_top_actors(10)

if __name__ == "__main__":
    main()
```

Required screenshots

[Insert authentic program-output screenshots showing both rankings limited to ten entries. Your name or FAU ID must be visible when captured and must not be added to the images.]

Problem 5 Weather station analyzer

p5_Alfaro_Jorge.py

```
"""Problem 5: weather observations, statistics and outliers.
```

```
Input is headerless CSV: station,date,temperature. Dates use
%I:%M:%S %p %m/%d/%Y. Output CSV contains station,min,max,mean.
Run: python p5_Alfaro_Jorge.py observations.csv statistics.csv
"""
```

```
import csv
from datetime import datetime
import sys
```

```
DATE_FORMAT = "%I:%M:%S %p %m/%d/%Y"
Observations = dict[str, list[tuple[str, float]]]
Statistics = dict[str, tuple[float, float, float]]
```

```
def read_observations(filename: str) -> tuple[Observations, list[tuple[int, str]]]:
    """Read valid observations in chronological order and report bad lines.
```

```
    The first valid observation for a station and parsed date wins.
    File-access errors propagate to the caller.
    """
```

```
    observations, errors, seen = {}, [], set()
    with open(filename, encoding="utf-8") as file:
        for line_number, line in enumerate(file, 1):
            try:
                row = next(csv.reader([line], strict=True))
                if len(row) != 3:
                    raise ValueError("Expected station,date,temperature.")
                station, date, text = (field.strip() for field in row)
                if not station:
                    raise ValueError("Station cannot be empty.")
                timestamp = datetime.strptime(date, DATE_FORMAT)
                temperature = float(text)
                if not -100.0 <= temperature <= 150.0:
                    raise ValueError("Temperature must be between -100 and 150.")
                key = (station, timestamp)
                if key in seen:
                    raise ValueError("Duplicate station/date observation.")
                seen.add(key)
                observations.setdefault(station, []).append((date, temperature))
            except (ValueError, csv.Error) as error:
                errors.append((line_number, str(error)))
    for records in observations.values():
        records.sort(key=lambda item: datetime.strptime(item[0], DATE_FORMAT))
    return observations, errors
```

```
def station_statistics(observations: Observations) -> Statistics:
    """Map each nonempty station to its minimum, maximum and mean."""
    statistics = {}
    for station, records in observations.items():
        if records:
            temperatures = [temperature for _, temperature in records]
            statistics[station] = (min(temperatures), max(temperatures),
                                   sum(temperatures) / len(temperatures))
    return statistics
```

Problem 5 Weather station analyzer continued

p5_Alfaro_Jorge.py

```
def station_outliers(observations: Observations) -> dict[str, tuple[str, float, float]]:
    """Map stations whose latest value exceeds their mean to (date, value, mean).

    Observations must be chronologically sorted, as read_observations returns.
    """
    statistics = station_statistics(observations)
    return {station: (*records[-1], statistics[station][2])
            for station, records in observations.items()
            if records and records[-1][1] > statistics[station][2]}

def write_statistics(filename: str, statistics: Statistics) -> None:
    """Write stations alphabetically, with all numbers to one decimal place."""
    with open(filename, "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow(["station", "min", "max", "mean"])
        for station in sorted(statistics):
            writer.writerow([station, *(f"{value:.1f}"
                                         for value in statistics[station])])

def main() -> int:
    """Read command-line paths, report results and handle file-access errors."""
    print("Student: Jorge Alfaro | FAU ID: Z23697022 | Problem 5")
    if len(sys.argv) != 3:
        print(f"Usage: {sys.argv[0]} INPUT.csv OUTPUT.csv", file=sys.stderr)
        return 1
    try:
        observations, errors = read_observations(sys.argv[1])
        statistics = station_statistics(observations)
        print("Statistics (minimum, maximum, mean)")
        for station in sorted(statistics):
            values = ", ".join(f"{value:.1f}" for value in statistics[station])
            print(f"{station}: {values}")
        print("Outliers (date, temperature, mean)")
        outliers = station_outliers(observations)
        for station, (date, value, mean) in sorted(outliers.items()):
            print(f"{station}: {date}, {value:.1f}, {mean:.1f}")
        for line, message in errors:
            print(f"Rejected line {line}: {message}")
        write_statistics(sys.argv[2], statistics)
        return 0
    except (OSError, UnicodeError) as error:
        print(f"Could not access the weather data: {error}", file=sys.stderr)
        return 1

if __name__ == "__main__":
    sys.exit(main())
```

Problem 5 Unit tests

test_p5.py

```
"""Run with: python -m unittest -v test_p5.py (from this directory)."""

from contextlib import redirect_stderr, redirect_stdout
import importlib.util
import io
from pathlib import Path
from tempfile import TemporaryDirectory
import unittest
from unittest.mock import patch

# Locate the solution even after its student-name placeholders are replaced.
SOURCE = next(Path(__file__).parent.glob("p5_*.py"))
SPEC = importlib.util.spec_from_file_location("weather", SOURCE)
weather = importlib.util.module_from_spec(SPEC)
SPEC.loader.exec_module(weather)

class WeatherTests(unittest.TestCase):
    """Exercise input validation, calculations, output and error handling."""

    def setUp(self) -> None:
        """Create an isolated directory for each test."""
        self.folder = TemporaryDirectory()
        self.addCleanup(self.folder.cleanup)
        self.input = Path(self.folder.name) / "input.csv"
        self.output = Path(self.folder.name) / "output.csv"

    def read(self, content: str) -> tuple:
        """Write a fixture and return its parsed observations and errors."""
        self.input.write_text(content, encoding="utf-8")
        return weather.read_observations(str(self.input))

    def test_several_stations_and_negative_temperatures(self) -> None:
        """Keep valid observations for distinct stations, including negatives."""
        observations, errors = self.read(
            "North,09:00:00 AM 01/01/2026,-12.5\n"
            "South,09:00:00 AM 01/01/2026,20\n")
        self.assertEqual(errors, [])
        self.assertEqual(set(observations), {"North", "South"})
        self.assertEqual(observations["North"][0][1], -12.5)

    def test_chronological_sort(self) -> None:
        """Sort across years and noon by datetime rather than date text."""
        observations, _ = self.read(
            "A,01:00:00 PM 01/01/2026,3\n"
            "A,11:00:00 PM 12/31/2025,1\n"
            "A,09:00:00 AM 01/01/2026,2\n")
        self.assertEqual([value for _, value in observations["A"]], [1, 2, 3])

    def test_duplicate_observations(self) -> None:
        """Reject repeat station/date pairs and keep the first valid value."""
        observations, errors = self.read(
            "A,09:00:00 AM 01/01/2026,10\n"
            "A,09:00:00 AM 01/01/2026,30\n")
        self.assertEqual(len(observations["A"]), 1)
        self.assertEqual(observations["A"][0][1], 10)
        self.assertEqual(errors[0][0], 2)
```

Problem 5 Unit tests continued

test_p5.py

```
self.assertIn("Duplicate", errors[0][1])

def test_invalid_ranges_and_nonfinite_values(self) -> None:
    """Accept both range endpoints; reject outside values, NaN and infinity."""
    observations, errors = self.read(""".join(
        f"S{i},09:00:00 AM 01/01/2026,{value}\n"
        for i, value in enumerate([-100, 150, -100.1, 150.1, "nan", "inf"])))
    self.assertEqual(set(observations), {"S0", "S1"})
    self.assertEqual([line for line, _ in errors], [3, 4, 5, 6])

def test_malformed_lines(self) -> None:
    """Report blank lines, wrong fields, invalid dates and bad numbers."""
    observations, errors = self.read(
        "\nA,date\n,09:00:00 AM 01/01/2026,2\n"
        "A,09:00:00 AM 02/30/2026,2\n"
        "A,09:00:00 AM 01/01/2026,hot\n"
        "unterminated,date,3\n")
    self.assertEqual(observations, {})
    self.assertEqual([line for line, _ in errors], list(range(1, 7)))

def test_statistics_and_outliers(self) -> None:
    """Check exact statistics and strict greater-than outlier comparison."""
    observations = {"A": [("earlier", -10), ("later", 20)],
                    "B": [("earlier", 20), ("later", 10)],
                    "C": [("only", 5)]}
    self.assertEqual(weather.station_statistics(observations),
                      {"A": (-10, 20, 5), "B": (10, 20, 15), "C": (5, 5, 5)})
    self.assertEqual(weather.station_outliers(observations),
                      {"A": ("later", 20, 5)})

def test_sorted_output_and_one_decimal(self) -> None:
    """Write lexicographic station order and exactly one decimal digit."""
    weather.write_statistics(str(self.output),
                             {"Zulu": (1, 2, 1.5), "Alpha": (-5, 0, -2.5)})
    self.assertEqual(self.output.read_text(encoding="utf-8"),
                      "station,min,max,mean\nAlpha,-5.0,0.0,-2.5\n"
                      "Zulu,1.0,2.0,1.5\n")

def test_missing_files(self) -> None:
    """Propagate access errors from the reader and writer."""
    with self.assertRaises(FileNotFoundError):
        weather.read_observations(str(self.input))
    with self.assertRaises(FileNotFoundError):
        weather.write_statistics(str(self.input / "missing.csv"), {})

def test_empty_input(self) -> None:
    """Empty input produces empty results without calculation errors."""
    self.assertEqual(self.read(""), ({}, []))
    self.assertEqual(weather.station_statistics({}), {})
    self.assertEqual(weather.station_outliers({}), {})

def test_main_handles_file_errors_and_usage(self) -> None:
    """Return failure without a traceback for usage and access errors."""
    for arguments in [
        ["weather.py"],
        ["weather.py", str(self.input), str(self.output)]:
        with patch.object(weather.sys, "argv", arguments):
            with redirect_stdout(io.StringIO()), redirect_stderr(io.StringIO()):
```

Problem 5 Unit tests continued

test_p5.py

```
        self.assertEqual(weather.main(), 1)

def test_main_success(self) -> None:
    """Run the complete command-line workflow and verify its output."""
    self.read("A,09:00:00 AM 01/01/2026,12.5\n")
    with patch.object(weather.sys, "argv",
                      ["weather.py", str(self.input), str(self.output)]):
        with redirect_stdout(io.StringIO()) as terminal:
            self.assertEqual(weather.main(), 0)
    self.assertIn("A: 12.5, 12.5, 12.5", terminal.getvalue())
    self.assertTrue(self.output.exists())

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Required screenshots

[Insert authentic screenshots of every prompt and answer from the coding agent inside VSCode. Your name or FAU ID must be visible in the application and must not be added to the images.]

Supplied testif module

testif.py

```
# testif module

def testif(b, testname, msgOK="", msgFailed=""):
    """Function used for testing.
    param b: boolean, normally a tested condition: true if test passed, false otherwise
    param testname: the test name
    param msgOK: string to be printed if param b==True ( test condition true)
    param msgFailed: string to be printed if param b==False
    returns b
    """
    if b:
        print("Success: "+ testname + "; " + msgOK)
    else:
        print("Failed: "+ testname + "; " + msgFailed)
    return b
```